CareerLens

Job, Skill and Recruitment Management System

DBMS Experiential Learning - Level 3 Project Definition

Department of Computer Science and Engineering, PES University

Section no.	Reg. no.	Team member	Project title
19	PES1UG24CS398	Sai Harshith R	CareerLens - Job, Skill and Recruitment Management System
19	PES1UG24CS394	Saathvik Shenoy Padubidri	(unique title within the section)

Stack: Next.js (UI and Route Handlers), PostgreSQL 16 with pgvector (Option 2: SQL + Vector database), Git + GitHub. **Roles:** Candidate, Recruiter.

1. Project Overview

Problem. Hiring is noisy on both sides. Candidates apply without knowing how well they fit a role or which skills they lack. Recruiters receive piles of unstructured resumes and filter by keywords, which misses good candidates who describe the same skills in different words.

Solution. CareerLens stores structured data (skills, proficiency, weighted job requirements, applications) in PostgreSQL and unstructured text (resumes and job descriptions) as vector embeddings in pgvector. Every candidate-job pair gets a **hybrid match score** that blends a weighted skill-overlap score computed in SQL with a semantic similarity score from vector search. Candidates see ranked jobs and the exact skills they are missing. Recruiters get ranked applicants, a managed hiring pipeline with an automatic audit trail, and a funnel dashboard.

Why this is beyond CRUD (Level 3). The core value is computed and not just stored: match scoring in a stored function, semantic ranking through vector search, skill-gap analysis, transactional applications, trigger-based audit history, and aggregate funnel analytics.

Objectives

- Design a normalized relational schema (ER diagram, relational schema, constraints) for a recruitment domain.
- Demonstrate DDL, DML, joins, grouping, subqueries, transactions, triggers, stored functions, views, indexing and full-text search in PostgreSQL.
- Demonstrate vector embeddings and similarity search with pgvector, and justify why a vector store is needed for resume and job text.
- Provide a secure, role-based Next.js web application that exercises all of the above.
- Maintain data integrity entirely at the database level.

Scope

In scope: registration and login, candidate profiles and skills, job posting with weighted skills, full-text job search with filters, hybrid job matching, skill-gap view, transactional apply, application pipeline with history, recruiter applicant ranking, and a recruiter funnel dashboard. **Out of scope:** resume file parsing (resume is entered as text), payments, email notifications, and a third (NoSQL) database.

2. Team Formation and Responsibilities

The project is built as a 2-member team. Responsibilities are split by module so that each member contributes to both the database and the application, and individual commits are visible in GitHub.

Module	Database work	Application work	Owner
Candidate side	candidates, candidate_skills, resume_embeddings; compute_skill_score(); skill-gap and match queries	Profile and skills pages, ranked jobs, skill gap, my applications	Member 1
Recruiter side	jobs, job_skills, job_embeddings; applications pipeline; status trigger; views; funnel queries	Post/edit/close job, ranked applicants, status change, funnel dashboard	Member 2
Shared	ER diagram, schema DDL, constraints, seed data, indexes	Authentication, role middleware, layout, README, documentation, testing	Both

3. Users and Roles

Two roles are implemented (minimum required: 2). The role is stored in users.role and enforced in backend middleware and in every Route Handler.

Role	Can do
Candidate	Register and log in; edit profile and resume text; add/update/remove skills with proficiency; search and filter jobs; view ranked job matches with match percentage; view skill gap for a job; apply to a job; withdraw an application; track own applications and status timeline.
Recruiter	Register and log in under a company; create, edit, close and delete own jobs with weighted mandatory/optional skills; view ranked applicants per job; change application status through the pipeline; view the recruiter funnel dashboard for own jobs.

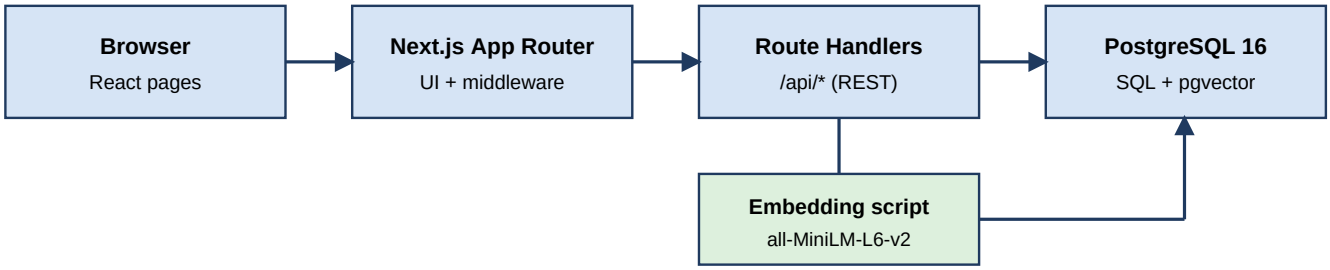
4. Functional Requirements

ID	Requirement
FR1	Users register and log in with email and password; passwords are hashed; a session/JWT carries the role.
FR2	Candidates manage profile, resume text and skills (proficiency 1-5, years used).
FR3	Recruiters create, edit, close and delete jobs, each with weighted required skills (weight 1-5, mandatory or optional), salary range, openings and deadline.
FR4	Anyone logged in can search jobs by keyword (full-text) and filter by location, salary and skill.
FR5	Candidates see open jobs ranked by hybrid match score (skills + semantic similarity).
FR6	For any job, a candidate sees the missing skills ordered by weight and which are mandatory.
FR7	Candidates apply to a job inside a database transaction; duplicates, closed jobs and passed deadlines are rejected.
FR8	Candidates track applications and the full status timeline; they can withdraw an application.
FR9	Recruiters view applicants for a job ranked by match score.
FR10	Recruiters move an application through the pipeline (applied, screening, interview, offer, hired, rejected); every change is logged automatically by a trigger.
FR11	Recruiters view a funnel dashboard (applications per status per job, conversion rates).
FR12	Resume and job embeddings are generated when a resume or job is created or edited.

5. Technology Stack and System Architecture

Layer	Technology
Frontend	Next.js (App Router), React, forms, tables, search, filters, dashboard pages
Backend / API	Next.js Route Handlers (REST under /api), middleware for auth and roles
Database	PostgreSQL 16 (relational data) with the pgvector extension (embeddings)
DB access	node-postgres (pg) with parameterized queries and a connection pool; plain SQL files for DDL, functions, triggers and views (no ORM)
Auth	bcrypt password hashing; JWT in an httpOnly cookie; role checks in middleware
Embeddings	all-MiniLM-L6-v2 (384 dimensions) via transformers.js or sentence-transformers
Version control	Git + GitHub with README and setup instructions

System architecture



Credentials and secrets are read from .env (DATABASE_URL, JWT_SECRET), never from source code.

Request flow example (candidate applies): the page calls POST /api/applications, the Route Handler checks the JWT and role, then calls the apply_to_job procedure inside a transaction; the match score is computed in SQL, the application and first history row are inserted, and the result is returned to the UI.

6. Database Choice and Justification

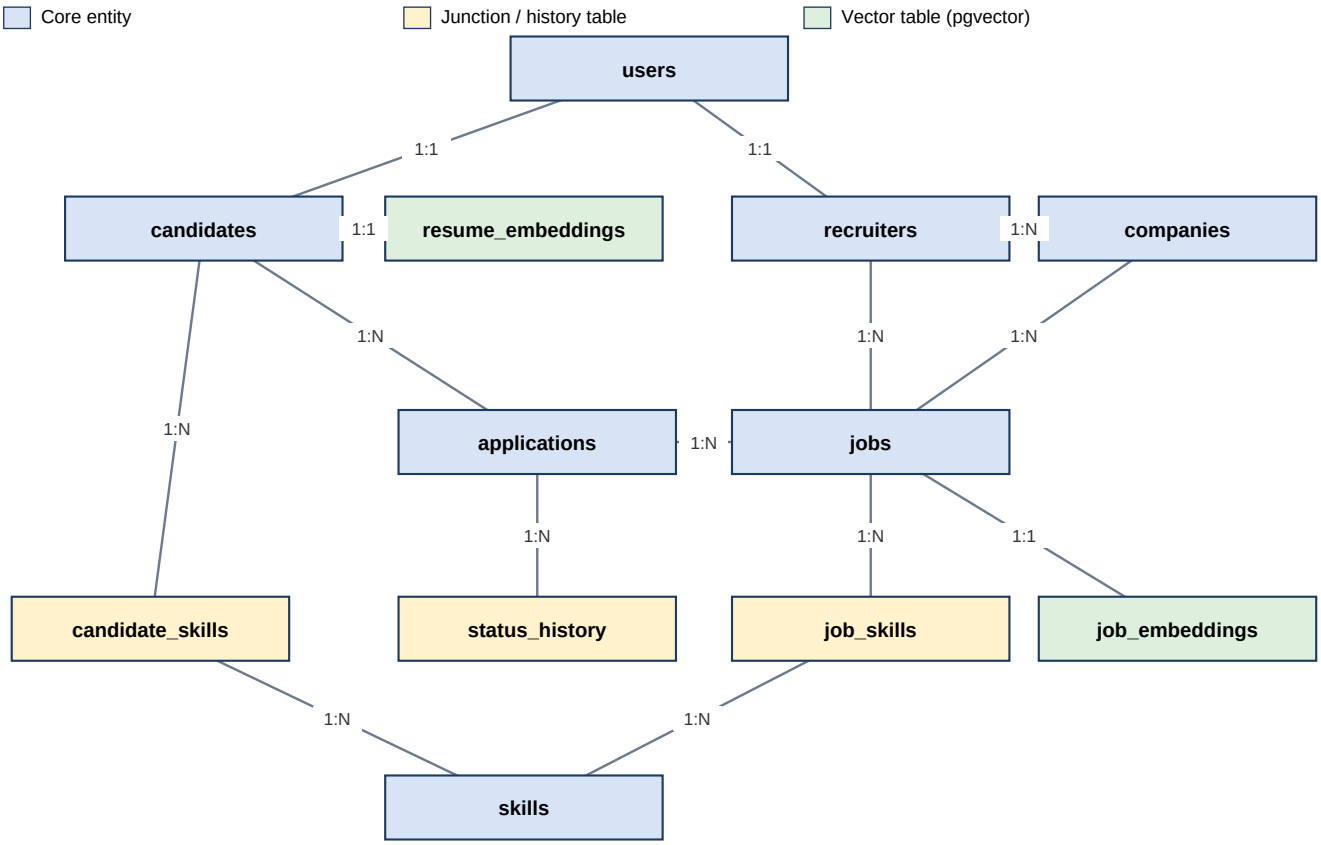
The project uses **Option 2: SQL + Vector database**, both inside PostgreSQL. The choice is driven by the nature of each type of data and not by technology demonstration.

Data	Store	Justification
Users, companies, candidates, recruiters, jobs, skills, applications, history	PostgreSQL (relational)	Highly structured, many-to-many relationships (candidate-skill, job-skill), needs foreign keys, constraints and ACID transactions (apply, status change). Joins and aggregates power matching and the funnel dashboard.
Resume text and job description embeddings	pgvector (vector(384), HNSW index, cosine)	Resumes and descriptions are unstructured text. Two documents can describe the same skills in different words, which SQL equality, LIKE and full-text keywords cannot capture. Vector similarity measures meaning.
Why not a separate NoSQL store	Not used	No data in this project is schema-less enough to need it. Resume text and metadata fit TEXT/columns. Keeping vectors in PostgreSQL keeps skill score and semantic score computable in one query, with no cross-database consistency problem.

7. Database Design

7.1 ER diagram

Entities, relationships and cardinalities. The final diagram is also exported as an image into docs/er-diagram in the repository.



7.2 Relationships

Relationship	Cardinality	Implemented by
User - Candidate / Recruiter	1 : 1	candidates.user_id and recruiters.user_id (UNIQUE FK)
Company - Recruiter	1 : N	recruiters.company_id FK
Company / Recruiter - Job	1 : N	jobs.company_id and jobs.recruiter_id FK
Candidate - Skill	M : N	candidate_skills (composite PK, proficiency, years_used)
Job - Skill	M : N	job_skills (composite PK, weight, is_mandatory)
Candidate - Job	M : N	applications (UNIQUE candidate_id + job_id, status, match_score)
Application - History	1 : N	application_status_history.application_id FK
Candidate / Job - Embedding	1 : 1	resume_embeddings and job_embeddings (PK = FK)

7.3 Relational schema and data dictionary

12 tables and 3 ENUM types (user_role: candidate, recruiter; job_status: open, closed; app_status: applied, screening, interview, offer, hired, rejected). All tables are in third normal form; M:N relationships are resolved with junction tables. PK = primary key, FK = foreign key.

users - Login identity and role.

Column	Type	Key / constraints
user_id	INT identity	PK
email	VARCHAR(255)	UNIQUE, NOT NULL, CHECK (email LIKE '%_@_%._%')
password_hash	VARCHAR(255)	NOT NULL (bcrypt hash)
role	user_role	NOT NULL
created_at	TIMESTAMPTZ	DEFAULT now()

companies - Hiring organizations.

Column	Type	Key / constraints
company_id	INT identity	PK
name	VARCHAR(150)	UNIQUE, NOT NULL
industry, location, website	VARCHAR	optional

candidates - Candidate profile; resume_text is the source for the resume embedding.

Column	Type	Key / constraints
candidate_id	INT identity	PK
user_id	INT	FK users, UNIQUE, NOT NULL, ON DELETE CASCADE
full_name	VARCHAR(120)	NOT NULL
phone	VARCHAR(15)	CHECK (phone ~ '^[0-9+ -]{7,15}$')
location, education	VARCHAR	optional
years_experience	NUMERIC(3,1)	DEFAULT 0, CHECK (>= 0)
summary, resume_text	TEXT	resume_text NOT NULL
updated_at	TIMESTAMPTZ	DEFAULT now()

recruiters - Recruiter profile linked to a company.

Column	Type	Key / constraints
recruiter_id	INT identity	PK
user_id	INT	FK users, UNIQUE, NOT NULL, ON DELETE CASCADE
company_id	INT	FK companies, NOT NULL, ON DELETE RESTRICT
full_name	VARCHAR(120)	NOT NULL
designation	VARCHAR(80)	optional

skills - Skill catalogue (seeded).

Column	Type	Key / constraints
skill_id	INT identity	PK
name	VARCHAR(80)	UNIQUE, NOT NULL
category	VARCHAR(50)	NOT NULL (e.g. Web, Data, DevOps, Database, Soft)

candidate_skills - Skills a candidate has.

Column	Type	Key / constraints
candidate_id	INT	FK candidates, ON DELETE CASCADE; part of PK
skill_id	INT	FK skills; part of PK
proficiency	SMALLINT	NOT NULL, CHECK (1 to 5)
years_used	NUMERIC(3,1)	DEFAULT 0, CHECK (>= 0)

jobs - Job postings.

Column	Type	Key / constraints
job_id	INT identity	PK
recruiter_id	INT	FK recruiters, NOT NULL
company_id	INT	FK companies, NOT NULL
title	VARCHAR(150)	NOT NULL
description	TEXT	NOT NULL (source for job embedding)
location	VARCHAR(100)	optional
salary_min, salary_max	INT	CHECK (salary_min >= 0 AND salary_min <= salary_max)
openings	SMALLINT	DEFAULT 1, CHECK (> 0)
deadline	DATE	NOT NULL (not in the past at creation, see trigger)
status	job_status	DEFAULT 'open'
created_at	TIMESTAMPTZ	DEFAULT now()
search_vector	TSVECTOR	GENERATED from title + description (added with ALTER TABLE)

job_skills - Weighted skills a job requires.

Column	Type	Key / constraints
job_id	INT	FK jobs, ON DELETE CASCADE; part of PK
skill_id	INT	FK skills; part of PK
weight	SMALLINT	NOT NULL, CHECK (1 to 5)
is_mandatory	BOOLEAN	NOT NULL DEFAULT false

applications - A candidate's application to a job.

Column	Type	Key / constraints
application_id	INT identity	PK
candidate_id	INT	FK candidates, NOT NULL, ON DELETE CASCADE
job_id	INT	FK jobs, NOT NULL, ON DELETE CASCADE
status	app_status	NOT NULL DEFAULT 'applied'
match_score	NUMERIC(5,2)	CHECK (0 to 100), computed at apply time
applied_at	TIMESTAMPTZ	DEFAULT now()
(candidate_id, job_id)	-	UNIQUE: blocks duplicate applications

application_status_history - Audit trail written by trigger.

Column	Type	Key / constraints
history_id	INT identity	PK
application_id	INT	FK applications, NOT NULL, ON DELETE CASCADE
old_status	app_status	NULL for the first row
new_status	app_status	NOT NULL
changed_at	TIMESTAMPTZ	DEFAULT now()

resume_embeddings - Vector of a candidate's resume.

Column	Type	Key / constraints
candidate_id	INT	PK and FK candidates, ON DELETE CASCADE
embedding	vector(384)	NOT NULL
model_name	VARCHAR(60)	DEFAULT 'all-MiniLM-L6-v2'
updated_at	TIMESTAMPTZ	DEFAULT now()

job_embeddings - Vector of a job description.

Column	Type	Key / constraints
job_id	INT	PK and FK jobs, ON DELETE CASCADE
embedding	vector(384)	NOT NULL
model_name	VARCHAR(60)	DEFAULT 'all-MiniLM-L6-v2'
updated_at	TIMESTAMPTZ	DEFAULT now()

8. SQL Implementation (DDL)

All DDL is stored in numbered files under db/ so the database can be rebuilt from scratch.

DDL element	Where it is demonstrated
CREATE	CREATE TYPE (3 enums), CREATE EXTENSION vector, CREATE TABLE (12), CREATE INDEX, CREATE VIEW, CREATE FUNCTION/PROCEDURE, CREATE TRIGGER
ALTER	ALTER TABLE jobs ADD COLUMN search_vector (generated tsvector); ALTER TABLE ... ADD CONSTRAINT for CHECKs added after initial design; ALTER TABLE applications ADD COLUMN match_score in a migration file
DROP	db:reset script drops and recreates the schema; DROP VIEW / DROP INDEX used when redefining the view and comparing query plans
Primary / foreign keys	Every table has a PK; 14 FKs with explicit ON DELETE rules
Constraints	NOT NULL, UNIQUE, CHECK, DEFAULT, ENUM (see section 15)
Indexes	B-tree on FKs and filter columns, composite (job_id, status), GIN on search_vector, HNSW on both embedding columns
Views	v_job_funnel, v_ranked_applicants

```
-- ALTER example: full-text column added after the first schema
ALTER TABLE jobs ADD COLUMN search_vector tsvector
  GENERATED ALWAYS AS (to_tsvector('english', title || ' ' || description)) STORED;

-- Index examples
CREATE INDEX idx_jobs_search      ON jobs USING GIN (search_vector);
CREATE INDEX idx_apps_job_status  ON applications (job_id, status);
CREATE INDEX idx_job_emb_hnsw     ON job_embeddings    USING hnsw (embedding vector_cosine_ops);
CREATE INDEX idx_resume_emb_hnsw  ON resume_embeddings USING hnsw (embedding vector_cosine_ops);
```

9. Database Operations (DML) and Important Queries

DML	Where it is used
INSERT	Register user, create profile, add skill, post job + job_skills, apply, history rows, embeddings, seed scripts
UPDATE	Edit profile/resume, change proficiency, edit job, close job, change application status (fires trigger), refresh embeddings
DELETE	Remove a skill from a profile, withdraw an application, delete a job (cascades to job_skills, applications, embeddings)
SELECT	Job search, ranked matches, skill gap, ranked applicants, funnel, dashboards

Important queries

#	Query purpose	Concepts shown
Q1	Ranked jobs for a candidate (hybrid score)	JOIN, function call, vector distance, ORDER BY, LIMIT
Q2	Skill gap: required skills the candidate lacks, by weight	LEFT JOIN / NOT EXISTS subquery, ORDER BY
Q3	Ranked applicants for a job	JOIN, ORDER BY on match_score (view)
Q4	Recruiter funnel: applications per status per job	GROUP BY, COUNT FILTER, aggregates
Q5	Companies with more than 2 open jobs	GROUP BY, HAVING
Q6	Jobs paying above the average salary	Scalar subquery, AVG
Q7	Full-text job search with rank	tsvector, plainto_tsquery, ts_rank
Q8	Average days from applied to hired	Join on history, aggregate, date arithmetic

```
-- Q1  Ranked jobs for candidate $1 (hybrid: 60% skills, 40% semantic)
WITH r AS (SELECT embedding FROM resume_embeddings WHERE candidate_id = $1)
SELECT j.job_id, j.title,
       compute_skill_score($1, j.job_id)               AS skill_score,
       100 * (1 - (je.embedding <=> r.embedding))      AS semantic_score
FROM jobs j JOIN job_embeddings je USING (job_id), r
WHERE j.status = 'open'
ORDER BY 0.6 * compute_skill_score($1, j.job_id)
       + 0.4 * 100 * (1 - (je.embedding <=> r.embedding)) DESC
LIMIT 10;

-- Q2  Skill gap for candidate $1 and job $2
SELECT s.name, js.weight, js.is_mandatory
FROM job_skills js JOIN skills s ON s.skill_id = js.skill_id
WHERE js.job_id = $2
  AND NOT EXISTS (SELECT 1 FROM candidate_skills cs
                  WHERE cs.candidate_id = $1 AND cs.skill_id = js.skill_id)
ORDER BY js.is_mandatory DESC, js.weight DESC;

-- Q5  Companies with more than 2 open jobs
SELECT c.name, COUNT(*) AS open_jobs
FROM jobs j JOIN companies c ON c.company_id = j.company_id
WHERE j.status = 'open'
GROUP BY c.name HAVING COUNT(*) > 2 ORDER BY open_jobs DESC;
```

10. Advanced Database Features

The guideline requires at least four advanced SQL concepts and one NoSQL/Vector concept. CareerLens implements six SQL concepts and the vector similarity search.

#	Feature	What it does in CareerLens	Demo
1	Transaction	apply_to_job: checks job is open and deadline not passed, computes match score, inserts application and first history row; rolls back on any failure	Apply to a closed job: nothing is written
2	Trigger	trg_status_history (AFTER UPDATE on applications) inserts a history row when status changes; trg_job_deadline (BEFORE INSERT on jobs) rejects past deadlines	Change status, show new history row
3	Stored function	compute_skill_score(candidate, job): weighted overlap of job skills the candidate has, halved if any mandatory skill is missing	Call it in psql
4	Views	v_job_funnel (counts per status per job), v_ranked_applicants	SELECT from view
5	Indexing	Composite, GIN and HNSW indexes; EXPLAIN ANALYZE compared before and after	Show plan change from Seq Scan to Index Scan
6	Full-text search	tsvector column + GIN index, ts_rank ordering for job search	Search 'react developer'
7	Vector similarity (vector DB)	Cosine similarity on 384-d embeddings with an HNSW index	Ranked jobs list (Q1)

```
-- Skill score: weighted share of the job's skills that the candidate has (0-100)
CREATE FUNCTION compute_skill_score(p_cand INT, p_job INT) RETURNS NUMERIC AS $$
  SELECT COALESCE(ROUND(
    100.0 * SUM(js.weight) FILTER (WHERE cs.skill_id IS NOT NULL) / NULLIF(SUM(js.weight), 0)
    * CASE WHEN BOOL_OR(js.is_mandatory AND cs.skill_id IS NULL) THEN 0.5 ELSE 1 END, 2), 0)
  FROM job_skills js
  LEFT JOIN candidate_skills cs ON cs.skill_id = js.skill_id AND cs.candidate_id = p_cand
  WHERE js.job_id = p_job;
$$ LANGUAGE sql STABLE;

-- Trigger: audit every status change
CREATE FUNCTION log_status_change() RETURNS trigger AS $$
BEGIN
  IF NEW.status IS DISTINCT FROM OLD.status THEN
    INSERT INTO application_status_history(application_id, old_status, new_status)
    VALUES (NEW.application_id, OLD.status, NEW.status);
  END IF;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_status_history AFTER UPDATE ON applications
FOR EACH ROW EXECUTE FUNCTION log_status_change();
```

11. Vector Database and Semantic Search

Resumes and job descriptions are unstructured, text-heavy data, which is where the guideline recommends vector search. The five required explanations:

Question	CareerLens answer
What data is converted into embeddings?	Candidate resume text (summary + resume_text + skill names) and job text (title + description + required skill names).
Which embedding model?	all-MiniLM-L6-v2 sentence-transformer, 384 dimensions, generated locally (no paid API).
How are vectors stored?	pgvector columns of type vector(384) in resume_embeddings and job_embeddings, same PostgreSQL database, with metadata (model_name, updated_at) and an HNSW index using vector_cosine_ops.
Which similarity metric?	Cosine similarity. pgvector's <=> operator returns cosine distance; similarity = 1 - distance.
How are relevant results retrieved?	The candidate's resume vector is compared with job vectors (top-K nearest neighbours via HNSW). The semantic score is blended with the SQL skill score: final = 0.6 x skill_score + 0.4 x semantic_score. Recruiters use the reverse direction (job vector against applicant resume vectors).

Vector data model and retrieval strategy

- Documents: one embedding per candidate and one per job (1:1 with the source row, same primary key).
- Metadata: model_name and updated_at; job status/deadline live in the relational table and are applied as SQL filters in the same query.
- Refresh: embeddings are regenerated when resume_text or job description changes.
- Embeddings are normalized so cosine and dot-product ranking agree.

12. Web Application Interface (Next.js)

Page	Role	UI components	Database functionality
/login, /register	All	Forms with validation	users insert, bcrypt check
/jobs	Both	Search box, filters (location, salary, skill), results table	Full-text search, filters
/jobs/[id]	Both	Job detail, skills with weights; candidate sees match % and skill gap	Hybrid score, skill-gap query
/matches	Candidate	Ranked job cards with score breakdown	Q1, vector search
/profile	Candidate	Profile form, skills table with add/update/delete	DML on candidates, candidate_skills; embedding refresh
/applications	Candidate	Application list, status timeline, withdraw button	History join, DELETE
/recruiter/jobs	Recruiter	Own jobs table; create/edit/close/delete job form with skill picker	DML on jobs, job_skills
/recruiter/jobs/[id]/applicants	Recruiter	Applicants ranked by score, status dropdown	v_ranked_applicants, UPDATE status (trigger)
/recruiter/dashboard	Recruiter	Funnel table and bar chart	v_job_funnel, aggregates

13. Database-Application Integration

The Next.js app communicates with PostgreSQL only through server-side Route Handlers. The browser never talks to the database. All queries are parameterized to prevent SQL injection.

Endpoint	Method	Access	Purpose
/api/auth/register, /login, /logout	POST	Public	Create account, issue/clear JWT cookie
/api/profile	GET, PUT	Candidate	Read/update profile and resume (refresh embedding)
/api/profile/skills	POST, PUT, DELETE	Candidate	Manage candidate_skills
/api/jobs	GET, POST	Both / Recruiter	Search jobs; create job with skills
/api/jobs/[id]	GET, PUT, DELETE	Both / Recruiter (owner)	View, edit, delete job
/api/matches	GET	Candidate	Ranked jobs (Q1)
/api/jobs/[id]/skill-gap	GET	Candidate	Skill gap (Q2)
/api/applications	GET, POST, DELETE	Candidate	List, apply (transaction), withdraw
/api/jobs/[id]/applicants	GET	Recruiter (owner)	Ranked applicants
/api/applications/[id]/status	PATCH	Recruiter (owner)	Change status (trigger logs it)
/api/dashboard/funnel	GET	Recruiter	Funnel data from view

Configuration: database credentials and secrets are never hard-coded. They are read from environment variables defined in .env (not committed) with a committed .env.example.

```
DATABASE_URL=postgres://user:password@localhost:5432/careerlens
JWT_SECRET=change-me
EMBEDDING_MODEL=Xenova/all-MiniLM-L6-v2
```

14. Authentication and Authorization

- Authentication is implemented in the backend: registration hashes passwords with bcrypt; login verifies the hash and issues a signed JWT stored in an httpOnly cookie.

- Authorization: Next.js middleware blocks unauthenticated access; each Route Handler checks the role (candidate or recruiter) and resource ownership (a recruiter can only touch their own jobs and applicants; a candidate only their own profile and applications).
- Two roles are implemented, meeting the minimum of 2.

15. Data Validation and Integrity

Validation is implemented at the database level (the UI validates too, but the database is the final authority).

Rule	Mechanism
No duplicate application to the same job	UNIQUE (candidate_id, job_id) on applications
Unique email / skill name / company name	UNIQUE constraints
Salary range valid, openings positive	CHECK (salary_min <= salary_max), CHECK (openings > 0)
Proficiency and weight between 1 and 5	CHECK constraints on candidate_skills and job_skills
Match score between 0 and 100	CHECK on applications.match_score
Status values limited	ENUM types app_status, job_status, user_role
Deadline not in the past on creation	BEFORE INSERT trigger on jobs
Apply only to open, unexpired jobs	Checked inside the apply_to_job transaction
No orphan rows	FKs with ON DELETE CASCADE (children) and RESTRICT (company with recruiters)
Every status change recorded	AFTER UPDATE trigger on applications
Atomic multi-step operations	Transactions (apply, create job with skills)

16. Seed Data Plan

A seed script (fixed random seed so both members get identical data) fills the database so the demo and analytics queries return meaningful results. Files run in order: schema, functions/triggers/views, reference seed, generated seed, embeddings.

Data	Approx. rows	Method
Skills	25	Hand-written SQL across Web, Data, DevOps, Database and Soft-skill categories
Companies / recruiters	4 / 4	Hand-written SQL
Jobs + job_skills	15 jobs, 4-6 skills each	Templates for 6 role archetypes with realistic descriptions (embeddings need real text)
Candidates + candidate_skills	20	Script; skills drawn from a target archetype so profiles are plausible; a few strong and weak fits for the demo
Applications + history	40	Script; funnel-shaped status distribution with spread-out timestamps
Embeddings	35	Script after text exists; HNSW index created after bulk insert

17. Testing Plan and Test Cases

ID	Test case	Expected result
T1	Register with duplicate email	Rejected by UNIQUE constraint
T2	Login with wrong password	401, no session
T3	Candidate opens a recruiter URL	403 / redirect
T4	Recruiter edits another recruiter's job	403, no change
T5	Create job with salary_min > salary_max	Rejected by CHECK
T6	Create job with a past deadline	Rejected by trigger
T7	Set proficiency = 7	Rejected by CHECK
T8	Apply twice to the same job	Second attempt rejected by UNIQUE
T9	Apply to a closed job or after deadline	Transaction rolls back, nothing inserted
T10	Change application status	Application updated and one history row created
T11	compute_skill_score for a known candidate and job	Matches the hand-calculated value
T12	Ranked matches for a known candidate	Closest-archetype jobs appear first
T13	Full-text search 'react'	Relevant jobs returned, ordered by ts_rank
T14	EXPLAIN ANALYZE job search before/after GIN index	Seq Scan replaced by index scan
T15	Delete a job	job_skills, applications and embeddings removed (cascade)
T16	Search repository for hard-coded credentials	None; all from .env

18. Git/GitHub

- Source code is maintained in a GitHub repository; each member commits their own modules with clear messages so individual contributions are visible.
- README includes prerequisites (Node, PostgreSQL 16, pgvector), .env setup, database setup (npm run db:reset), seeding, and how to run the app.

```
careerlens/
  db/        01_schema.sql  02_functions.sql  03_triggers.sql  04_views.sql  05_indexes.sql  seed/
  app/       (Next.js pages)   app/api/ (Route Handlers)   middleware.ts
  lib/       db.ts (pg pool)   auth.ts   embeddings.ts
  docs/      er-diagram  data-dictionary  queries  screenshots  test-results
  .env.example   README.md
```

19. Project Documentation (Soft Copy)

Required item	Source in this project
Objectives	Section 1
Functional requirements	Section 4
ER diagram	Section 7.1 and docs/er-diagram
Relational schema	Section 7.3
Database choice and justification	Section 6
Data dictionary / schema description	Section 7.3
SQL / vector database implementation	Sections 8, 10, 11 and db/ folder
Important queries	Section 9 and docs/queries
System architecture	Section 5
Application screenshots	Captured from final app into docs/screenshots
Testing results	Section 17 executed and recorded in docs/test-results
Conclusion	Written after implementation

20. Evaluation and Demonstration Plan

Step	Demonstration	Guideline item shown
1	ER diagram, relational schema, database justification	Database design
2	Run schema and seed; show tables, constraints, indexes	Database implementation
3	Recruiter logs in, posts a job with weighted skills	Next.js interface, DML, integration
4	Candidate logs in, sees ranked matches and skill gap	Vector search, function, queries
5	Candidate applies; a second attempt is rejected	Transaction, UNIQUE constraint
6	Recruiter changes status; history row appears	Trigger
7	Funnel dashboard; job search; EXPLAIN before/after index	View, full-text, indexing
8	Run the test cases	Test cases
9	Each member explains their modules and GitHub commits	Individual contribution

21. Originality and Level 3 Expectation

- The problem statement, hybrid scoring formula, schema, queries and features are designed specifically for this project and are not copied from an existing project, tutorial or repository.
- The project goes beyond CRUD: computed match scores, vector similarity ranking, skill-gap analysis, transactions, triggers and aggregate analytics.
- Database technologies are justified by data type: PostgreSQL for structured, relational, transactional data; pgvector for unstructured resume and job text.

22. Compliance Matrix (Guideline Checklist)

#	Guideline	How CareerLens satisfies it	Section
1	Unique title, real-world problem, not simple CRUD	CareerLens; hiring problem; computed matching	1
2	2-member team, defined responsibilities in DB and app	Module split, both touch DB + app	2
3	SQL (MySQL/PostgreSQL) + NoSQL or Vector	PostgreSQL + pgvector (Option 2)	6
4	ER diagram, relational schema, NoSQL/vector documentation	12 tables, keys, constraints; vector model documented	7, 11
5	DDL: CREATE, ALTER, DROP, keys, constraints, indexes, views	Full DDL files	8
6	DML and JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subqueries	Queries Q1-Q8	9
7	4 advanced SQL + 1 NoSQL/Vector	6 SQL features + vector similarity	10
8	Vector DB / semantic search explained (5 points)	Table of five answers	11
9	Next.js web interface with forms, tables, search, dashboards	Pages table	12
10	Backend/API layer, no hard-coded credentials, env variables	Route Handlers, .env	13
11	Authentication in backend, at least 2 roles	JWT + bcrypt, Candidate and Recruiter	14
12	Database-level validation and integrity	Constraints, triggers, transactions	15
13	Documentation (soft copy)	Mapped item by item	19
14	Git/GitHub with individual commits and README	Repo structure and README	18
15	Demonstration contents	Demo plan and test cases	17, 20
16	Originality	Statement and distinct design	21
17	Level 3: beyond CRUD, justified DB choices	Sections 6, 10, 21	6, 10, 21